

u24916031 - Jamie King
u25089464 - Lushomo Mankomba
u24984362- Joelle Tsaku

[Github Repo Link](#)

Taskforge is a parcel delivery management system, aimed to help keep track of parcels that are to be delivered to customers. The system keeps track of the state of each parcel and allows retrieving parcels based on priority delivery.

Task 1: Designing the system

- Domain: regional courier parcel delivery.
- Problem: a courier network organizes deliveries hierarchically (country - province -city - zip code -parcel), but different staff need to inspect that same structure differently (a full manifest vs a priority only queue); individual parcels move through a real delivery lifecycle where invalid actions must be rejected sensibly and some parcels need extra handling (signature, fragile care, priority) added without bloating every parcel with unused fields

Iterator:

Iterator - Iterator
ConcreteIterator - ParcelIterator
Aggregate - Aggregate
ConcreteAggregate - ZipCode

Composite:

Component - AreaComponent
Leaf - ZipCode
Composite - Region

State:

Context - Parcel
State - ParcelState
ConcreteStateA - AwaitingPickupState
ConcreteStateB - OnRouteState
ConcreteStateC - DeliveredState
ConcreteStateD - LostState

Decorator:

Component - Parcel
ConcreteComponent - ConcreteParcel
Decorator - Decorator
ConcreteDecoratorA - Signed
ConcreteDecoratorB - Fragile
ConcreteDecoratorC - Priority
ConcreteDecoratorD - Large

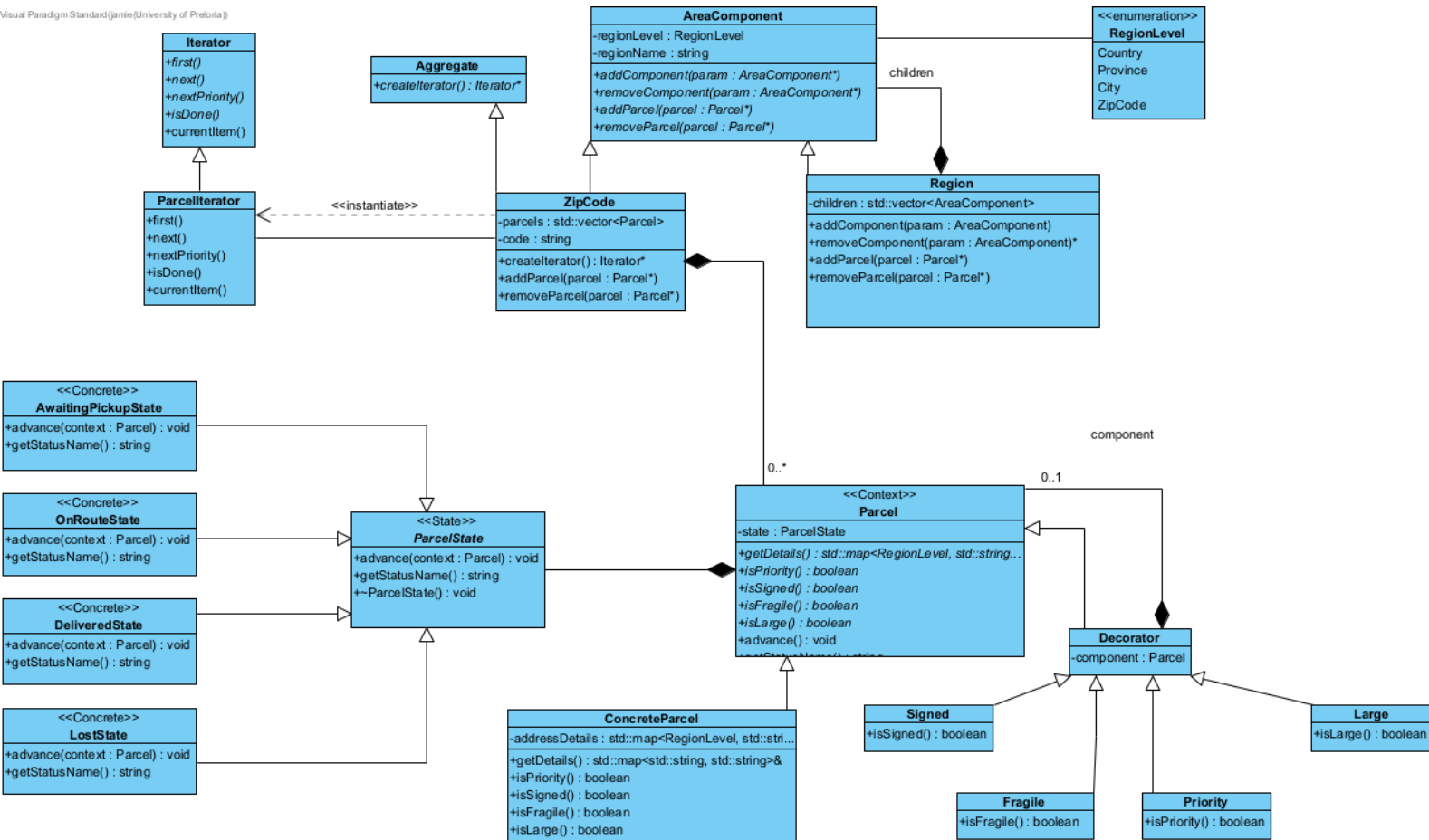
Design Rationale:

In order to have multiple traversals over the aggregate we allowed parcels to be priority. Whether a parcel is a priority is determined by its decorators. This is so we can either simply get the next parcel in the list or get the next parcel with priority.

Traversal Policy:

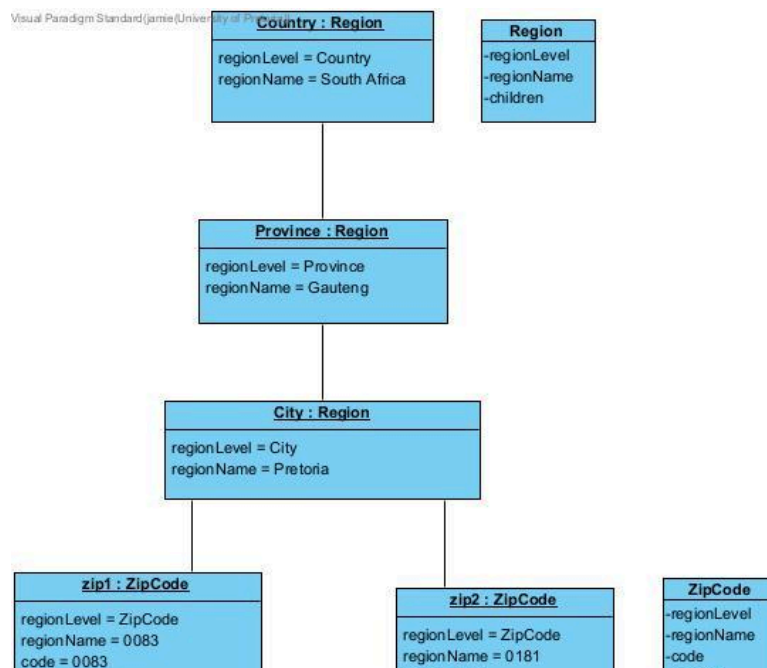
We create a snapshot of the traversal state from when the iterator is created. This means if the tree is modified the iterator remains unaffected. All ownership is composite to relieve the client of having to delete all the heap allocated objects.

Visual Paradigm Standard (jamie@University of Pretoria)

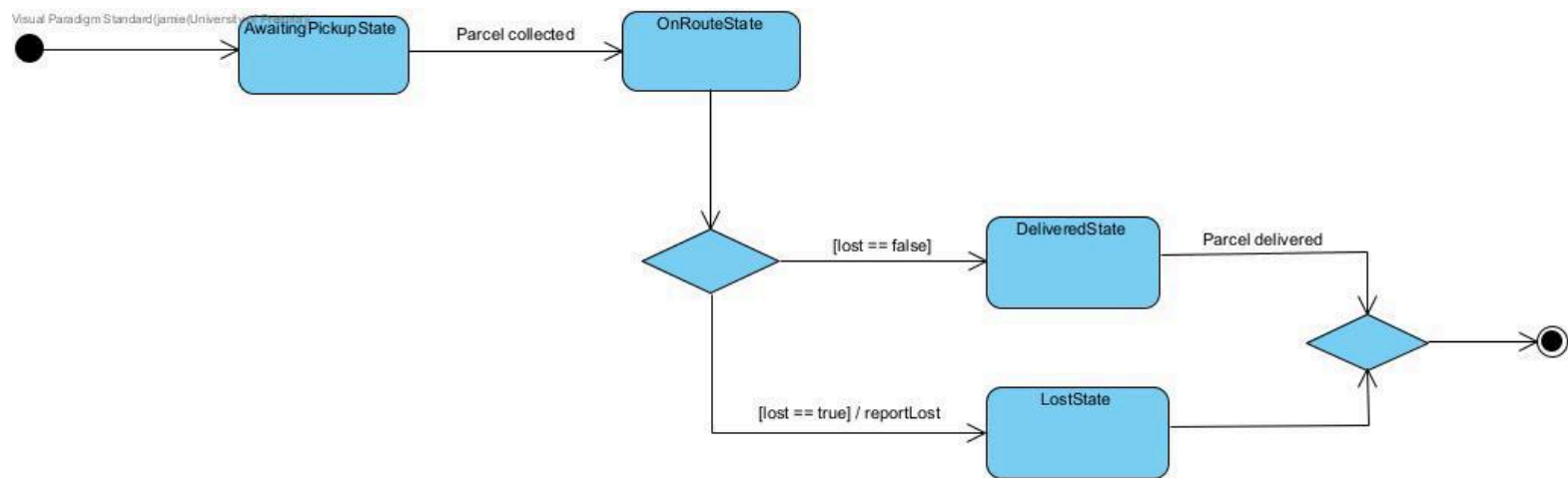


Object Diagram:

Visual Paradigm Standard (jamie@University of Pretoria)



State Diagram:



Activity Diagrams:

Diagram 1: Create Parcel

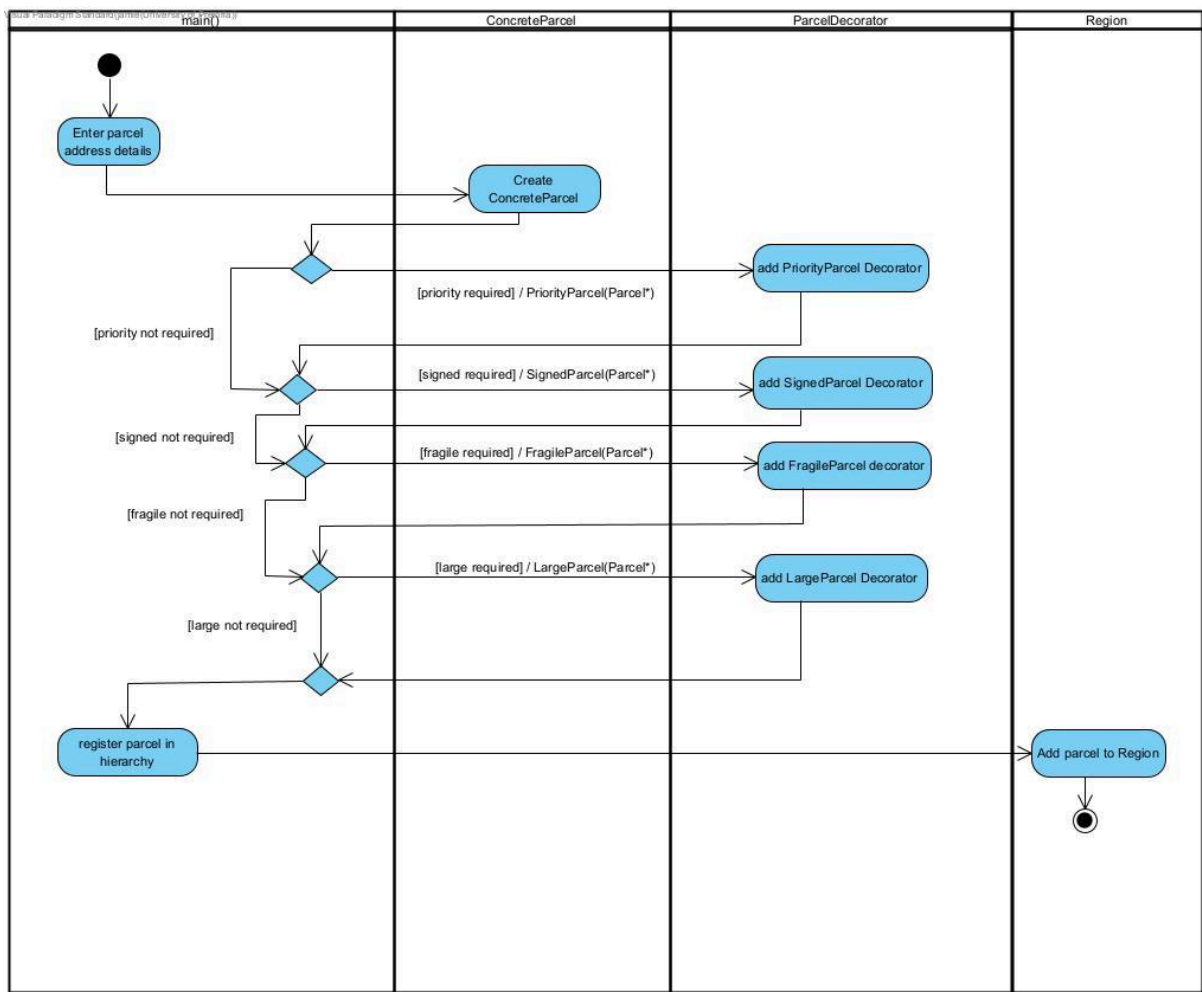


Diagram 2: Parcel State Lifecycle

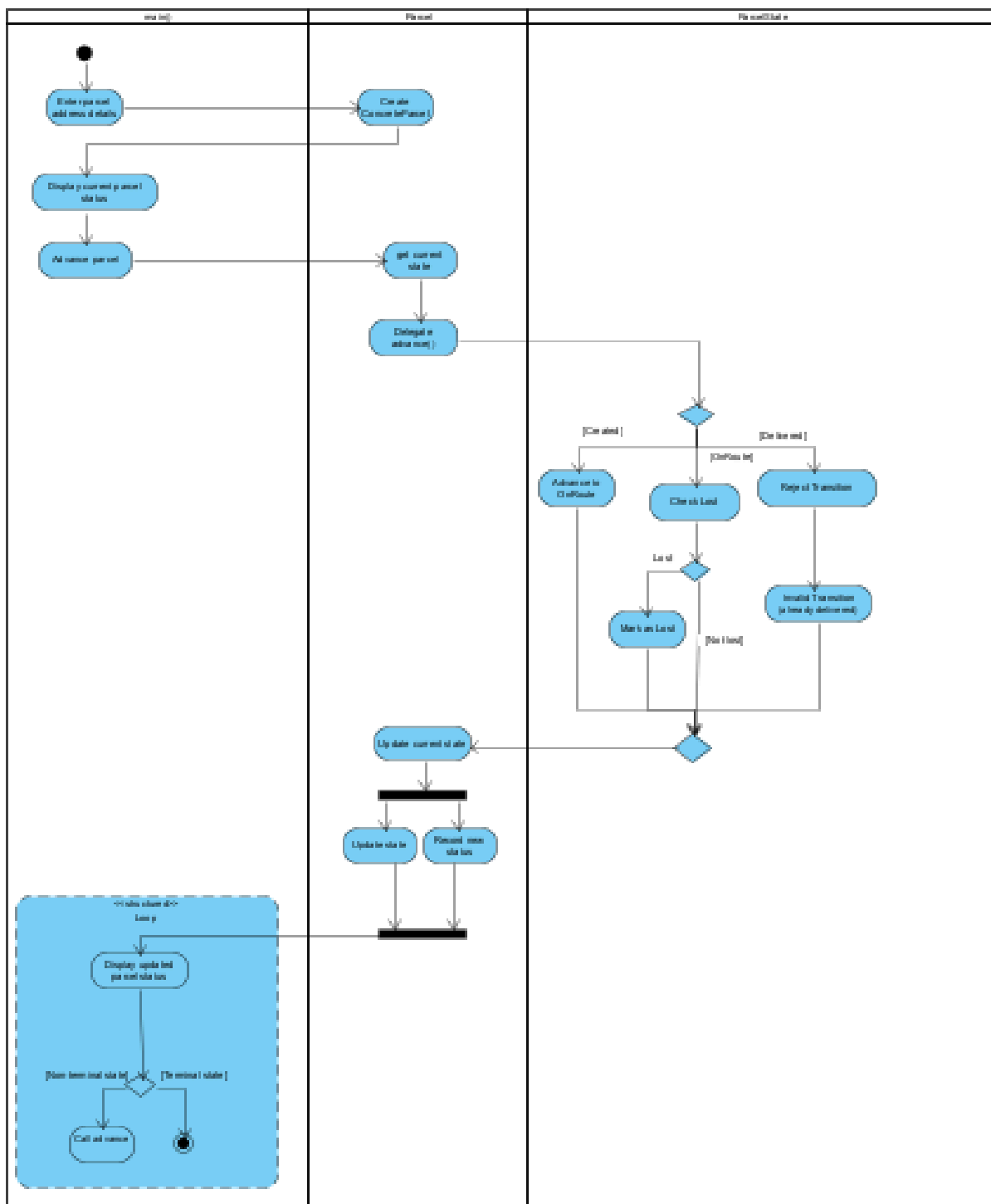
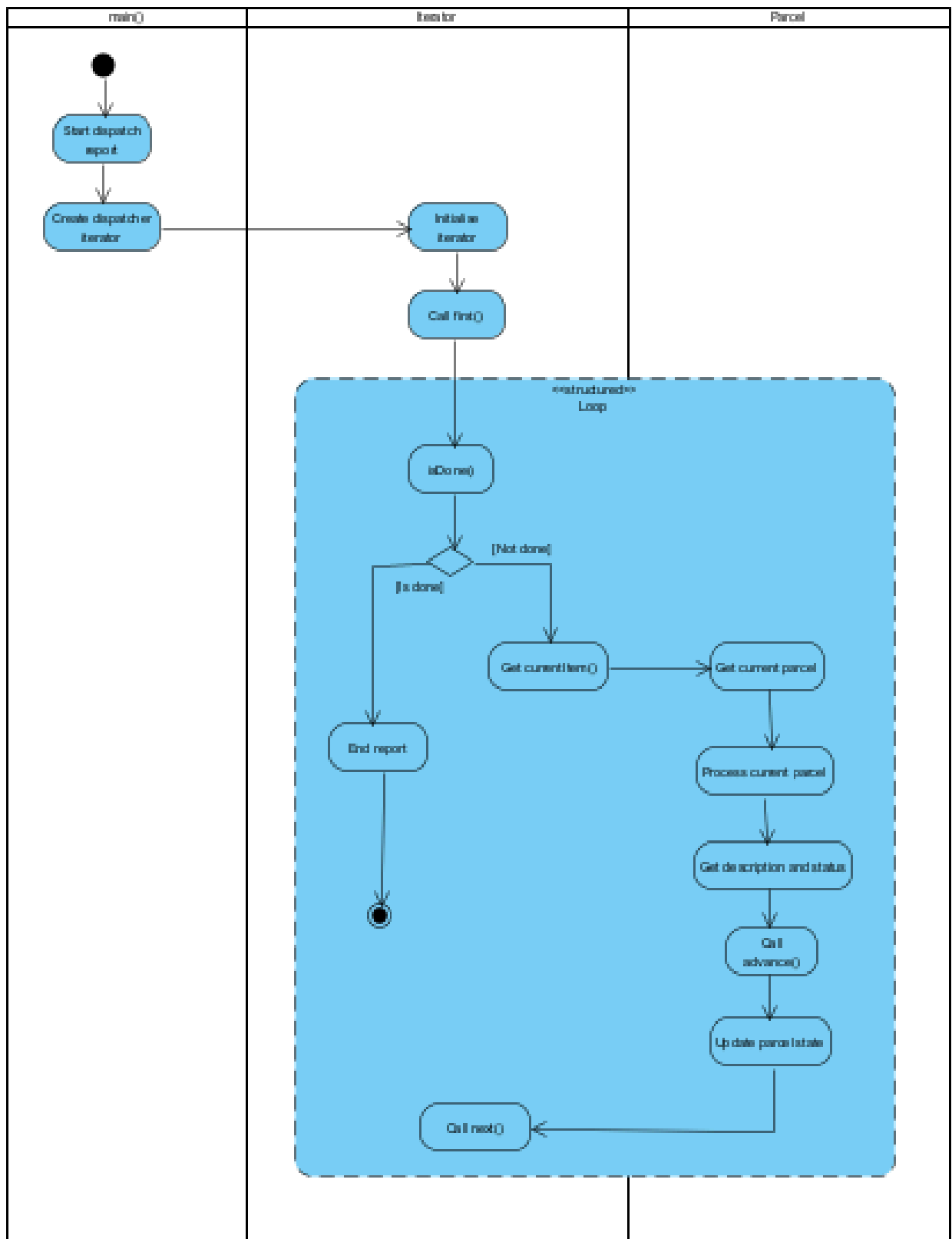


Diagram 3: Dispatcher Parcel Traversal



GDB Bug Investigation:

After creating our main function, when trying to compile the code we ran into a segfaulting error.

```
jamie@DESKTOP-I8P9Q9D:/mnt/c/Users/jamie/OneDrive/Documents/2026 UNI/SEM2/COS214/PRAC4$ make run
./taskforge
inval a ZipCode cannot contain nested regions
make: *** [Makefile:25: run] Segmentation fault (core dumped)
```

To find the error we used gdb:

```
jamie@DESKTOP-I8P9Q9D:/mnt/c/Users/jamie/OneDrive/Documents/2026 UNI/SEM2/COS214/PRAC4$ make runtest
gdb ./taskforgetest
GNU gdb (Ubuntu 15.0.50.20240403-0ubuntu1) 15.0.50.20240403-git
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./taskforgetest...
(gdb) break main
Breakpoint 1 at 0xbe79: file src/main.cpp, line 14.
(gdb) break 21
Breakpoint 2 at 0xc083: file src/main.cpp, line 21.
(gdb) break 32
Breakpoint 3 at 0xc1d0: file src/main.cpp, line 33.
(gdb) break 42
Breakpoint 4 at 0xc3cf: file src/main.cpp, line 43.
(gdb) break 48
Breakpoint 5 at 0xc506: file src/main.cpp, line 48.
(gdb) break 57
Breakpoint 6 at 0xc6bd: file src/main.cpp, line 57.
(gdb) break 64
Breakpoint 7 at 0xc863: file src/main.cpp, line 64.
(gdb) break 64
Note: breakpoint 7 also set at pc 0xc863.
Breakpoint 8 at 0xc863: file src/main.cpp, line 64.
(gdb) break 76
Breakpoint 9 at 0xca72: file src/main.cpp, line 76.
(gdb) break 84
Breakpoint 10 at 0xcb95: file src/main.cpp, line 84.
(gdb) break 95
Breakpoint 11 at 0xccdd: file src/main.cpp, line 95.
(gdb) break 106
Breakpoint 12 at 0xce65: file src/main.cpp, line 106.
(gdb) break 122
Undefined command: "break122". Try "help".
(gdb) break 122
Breakpoint 13 at 0xd07b: file src/main.cpp, line 122.
(gdb) break 137
Breakpoint 14 at 0xd22d: file src/main.cpp, line 137.
(gdb) r
Starting program: /mnt/c/Users/jamie/OneDrive/Documents/2026 UNI/SEM2/COS214/PRAC4/taskforgetest
```

This allowed us to find what function was causing the error, which was Region::addParcel()

```

Starting program: /mnt/c/Users/jamie/OneDrive/Documents/2026 UNI/SEM2/COS214/PRAC4/taskforgetest

This GDB supports auto-downloading debuginfo from the following URLs:
<https://debuginfod.ubuntu.com>
Enable debuginfod for this session? (y or [n]) n
Debuginfod has been disabled.
To make this setting permanent, add 'set debuginfod enabled off' to .gdbinit.
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, main () at src/main.cpp:14
14     int main() {
(gdb) c
Continuing.

Breakpoint 2, main () at src/main.cpp:21
21     ZipCode* zip0083 = new ZipCode("0083");
(gdb) c
Continuing.
inval a ZipCode cannot contain nested regions

Breakpoint 3, main () at src/main.cpp:33
33     {RegionLevel::Country, "South Africa"},
(gdb) c
Continuing.

Breakpoint 4, main () at src/main.cpp:43
43     {RegionLevel::Country, "South Africa"},
(gdb) c
Continuing.

Breakpoint 5, main () at src/main.cpp:48
48     ConcreteParcel* p3 = new ConcreteParcel(addr0083);
(gdb) c
Continuing.

Program received signal SIGSEGV, Segmentation fault.
0x000055555555a6f1 in Region::addParcel (this=0x55555557d2b0, parcel=0x55555557da50) at src/Region.cpp:31
31     if (details[(*it)->getLevel()] == (*it)->getName()) {
(gdb)

```

```

void Region::addParcel(Parcel* parcel) {
    auto details = parcel->getDetails();

    std::vector<AreaComponent*>::iterator it;
    for (; it != children.end(); it++) {
        if (details[(*it)->getLevel()] == (*it)->getName()) {
            (*it)->addParcel(parcel);
            return;
        }
    }
}

```

After taking a closer look at the function we realised that we weren't initialising the iterator, which was causing the issue

```

void Region::addParcel(Parcel* parcel) {
    auto details = parcel->getDetails();

    std::vector<AreaComponent*>::iterator it;
    for (;it != children.end();it++) {
        if (details[(*it)->getLevel()] == (*it)->getName()) {
            (*it)->addParcel(parcel);
            return;
        }
    }
}

```

```

void Region::addParcel(Parcel* parcel) {
    auto details = parcel->getDetails();

    std::vector<AreaComponent*>::iterator it = children.begin();
    for (;it != children.end();it++) {
        if (details[(*it)->getLevel()] == (*it)->getName()) {
            (*it)->addParcel(parcel);
            return;
        }
    }
}

```

After fixing that everything was working again, as intended.

```

jamie@DESKTOP-18P9Q9D:/mnt/c/Users/jamie/OneDrive/Documents/2026 UNIT/SEM2/COS214/PRACA$ make run
./taskforge
inval a ZipCode cannot contain nested regions
----- Decorator - stacked responsibilities -----
Standard parcel This parcel has to be signed for on delivery. This parcel is fragile. Handle with care. This parcel has priority, and should be dispatched as soon as possible.
priority=1 fragile=1 signed=1 large=0

----- State: p1 valid lifecycle -----
p1: Awaiting Pickup
p1: On Route
p1: Delivered

----- State with invalid transition -----
Parcel has already been delivered.

----- State with alternate branch (lost) -----
p2: Lost
Invalid transition: parcel is lost and cannot advance in delivery.

----- Iterator: full sweep vs priorit -----
Full sweep: Delivered | Lost | Awaiting Pickup | Awaiting Pickup |
Priority only: Standard parcel This parcel has to be signed for on delivery. This parcel is fragile. Handle with care. This parcel has priority, and should be dispatched as soon as possible. |

----- Scenario 1: dispatcher full day report -----
Standard parcel [Delivered]
Parcel has already been delivered.
Standard parcel [Lost]
Invalid transition: parcel is lost and cannot advance in delivery.
Standard parcel [Awaiting Pickup]
Standard parcel This parcel has to be signed for on delivery. This parcel is fragile. Handle with care. This parcel has priority, and should be dispatched as soon as possible. [Awaiting Pickup]

----- Scenario 2: delivery + structural change -----
Snapshot iterator (created before removal) still sees: Delivered | On Route |
New iterator (created after removal) sees 1 parcels left in zip 0003.
jamie@DESKTOP-18P9Q9D:/mnt/c/Users/jamie/OneDrive/Documents/2026 UNIT/SEM2/COS214/PRACA$

```

Valgrind Evidence:

```
jamie@DESKTOP-I8P9Q9D:/mnt/c/Users/jamie/OneDrive/Documents/2026 UNI/SEM2/COS214/PRAC4$ valgrind --leak-check=full --show-leak-kinds=all ./taskforge
==3219== Memcheck, a memory error detector
==3219== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==3219== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==3219== Command: ./taskforge
==3219==
inval a ZipCode cannot contain nested regions
----- Decorator - stacked responsibilities -----
Standard parcel This parcel has to be signed for on delivery. This parcel is fragile. Handle with care. This parcel has priority, and should be dispatched as soon as possible.
priority=1 fragile=1 signed=1 large=0

----- State: p1 valid lifecycle -----
p1: Awaiting Pickup
p1: On Route
p1: Delivered

----- State with invalid transition -----
Parcel has already been delivered.

----- State with alternate branch (lost) -----
p2: Lost
Invalid transition: parcel is lost and cannot advance in delivery.

----- Iterator: full sweep vs priorit -----
Full sweep: Delivered | Lost | Awaiting Pickup | Awaiting Pickup |
Priority only: Standard parcel This parcel has to be signed for on delivery. This parcel is fragile. Handle with care. This parcel has priority, and should be dispatched as soon as possible. |

----- Scenario 1: dispatcher full day report -----
Standard parcel [Delivered]
Parcel has already been delivered.
Standard parcel [Lost]
Invalid transition: parcel is lost and cannot advance in delivery.
Standard parcel [Awaiting Pickup]
Standard parcel This parcel has to be signed for on delivery. This parcel is fragile. Handle with care. This parcel has priority, and should be dispatched as soon as possible. [Awaiting Pickup]

----- Scenario 2: delivery + structural change -----
Snapshot iterator (created before removal) still sees: Delivered | On Route |
New iterator (created after removal) sees 1 parcels left in zip 0083.
==3219==
==3219== HEAP SUMMARY:
==3219==    in use at exit: 0 bytes in 0 blocks
==3219== total heap usage: 154 allocs, 154 frees, 83,914 bytes allocated
==3219==
==3219== All heap blocks were freed -- no leaks are possible
==3219==
==3219== For lists of detected and suppressed errors, rerun with: -s
==3219== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
jamie@DESKTOP-I8P9Q9D:/mnt/c/Users/jamie/OneDrive/Documents/2026 UNI/SEM2/COS214/PRAC4$
```

Task 3: Dynamic Behaviour and Design Decisions

- Scenario 1 (dispatchers full day report): iterates the whole network, prints each parcels status, shows full traversal and state display together.
- Scenario 2 (delivery and structural change): delivers a parcel, attempts an invalid rescan that gets rejected, reports a different parcel lost which changes state, then removes the delivered parcel from its ZipCode (structural change) — we compare a snapshot iterator taken before the removal against a fresh one taken after, to explicitly demonstrate the traversal modification steps
- our traversal modification policy: We chose snapshot traversal over live traversal because it avoids iterator invalidation or dangling pointers if a parcel is removed (lost) or archived mid-traversal, at the cost of a snapshot iterator not reflecting changes made after its creation this is fine though since each report is a point-in-time thing.

Task 4: UML Diagram Portfolio

- Diagram 1 (Create Parcel) shows decision/guard conditional decorator wrapping with merge points recombining each branch before the next check
- Diagram 2 (Parcel state lifecycle) shows the systems conditional, lifecycle-dependent behaviour in full. this is including the fork and join synchronising state update and status broadcast and demonstrates invalid-transition rejection ("Invalid Transition (already delivered)").
- Diagram 3 (Dispatcher parcel traversal) makes the Iterator traversal explicit as a first-class part of the dispatch workflow rather than collapsing it into a single "process all" step. All three diagrams use swimlanes to separate responsibility between main(), the pattern classes, and the Iterator itself.

Workflow reflection:

We initially had a meeting where we came up with our idea and fleshed out all the classes that we would have. One person made an initial Class diagram and Skeletons for all the classes we had. From here we would make a branch and flesh out all the classes for each design pattern involved before merging back into main.

Team Contribution Statement

Jamie King:

Created initial class diagram without state design pattern and skeletons of all the classes.
Implemented all the logic for the participants of the Composite design pattern.
Performed gdb debugging
Made the makefile, dockerfile and readme.
Made the Object diagram.
Compiled final PDF submission.

Lushomo Mankomba:

Added State design pattern to initial class diagram.
Implemented all the logic for the Iterator and State patterns
Implemented main and created the runtime scenarios

Joelle Tsaku:

Implemented all the logic for the Parcel Decorator
Designed the UML State Diagram and Activity Diagrams
Added .gitignore and .dockerignore file